Mahidol University
Wisdom of the Land

Mahidol University

Lecture 12: Function II

Lecturers:
Aj. Jidapa Kraisangka
Aj. Akara Supratak
Aj. Suradej Intagorn

Recap

parameters vs argument

parameter

```
def exponent(num,  power=2):
    return num ** power

print(exponent(2,3)) #8
print(exponent(3,2)) #9
print(exponent(3)) #9 --> default value for power
print(exponent())
```

argument

- parameter คือตัวแปรที่ถูกระบุไว้ใน function
- argument คือค่าที่ถูกส่งไปให้ function ซึ่งค่าที่ใส่ไปนี้จะถูกเก็บโดย parameter ตามลำดับ

Contact: [jidapa.kra, akara.sup, suradej.int]@mahidol.edu

default parameters

- default parameter มีไว้เพื่อกำหนดค่าเริ่มต้นใน function
- default parameter จะถูกเรียกใช้เมื่อ ไม่มีการส่งค่าไปที่ function นั้นๆ

default parameter: ลำดับของ default parameter ควรอยู่หลัง parameter ธรรมดา

```
def exponent(num, power=2):
    return num ** power

print(exponent(2,3)) #8
print(exponent(3,2)) #9
print(exponent(3)) #9 --> default value for power
print(exponent())
```

```
TypeError: exponent() missing 1
required positional argument: 'num'
```

default parameters


```
def add(a,b):
    return a+b
def math(a,b, fn=add): #default should go the last, order does matter
    return fn(a,b)
def subtract(a,b):
    return a-b

print(math(4,5))
print(math(4,5, subtract))
```

```
9
-1
```

Keyword Arguments

- เราสามารถระบุค่าของ arguments ที่ถูกส่งไปให้ parameters ใน function ได้
- ใน function เราสามารถตั้งค่า default parameter ได้
- เมื่อเรียกใช้ function เราสามารถตั้งค่า keyword argument ได้

```
def exponent(num,  power=2):
    return num ** power

print(exponent(num = 2,power = 3)) #8
print(exponent(power = 3, num = 2)) #8
print(exponent(2, 3)) #8
```

การเตรียมตัวสำหรับ Check Point 3

- สอบทฤษฎีในระบบ MyCourse และข้อสอบปฏิบัติ 2 ข้อ ใน PC2 เวลา 1 ชั่วโมง 30 นาที
- สอบเวลาประมาณ 14:30 - 16:00 น. (ไม่มี Lab)
- Closed book แต่สามารถนำ Cheat sheet ขนาด A4 หน้า-หลัง เข้าห้องสอบได้
- ห้ามนักศึกษาออกจากห้องสอบก่อนเวลา หากสอบเสร็จก่อนเวลา **ห้าม** เปิด Program อื่น ๆ หรือเครื่องมือสื่อสารขณะอยู่ในห้องสอบ

- สามารถอ่านคำชี้แจงการสอบได้ใน MyCourses

Contact: [jidapa.kra, akara.sup, suradej.int]@mahidol.edu

Today Objectives/ Topics

1. สามารถใช้ * และ ** เพื่อระบุ parameter เป็นชื่อเดียว และเพื่อเก็บ arguments ที่ระบุชื่อและค่าหลายชุดได้
2. เข้าใจว่า lambda, map, filter คืออะไรและใช้งานได้อย่างไร
3. สามารถใช้ built-in functions เช่น min, max, sort, reverse ได้

*args

- *args คือ Parameter ที่ใช้แทน Parameter ตามตำแหน่ง (ไม่มีชื่อ) กี่ตัวก็ได้ มี Type เป็น Tuple
- เราสามารถใช้ชื่อตัวแปรอื่นแทน args ได้ เช่น *tmp, *vars แต่โดยทั่วไป programmer จะใช้ชื่อ *args เป็นมาตรฐาน

*args

Before *args

```
def sum_all_nums(num1, num2, num3):
    return num1+num2+num3

print(sum_all_nums(1,2,3)) #6
```

```
def sum_all_nums(num1, num2, num3=0, num4=0, num5=0):
    return num1+num2+num3+num4+num5

print(sum_all_nums(1,2,3)) #6
```

*args

After *args

```
def sum_all_nums(*args):
    print(args)  #tuple of all parameters we pass in
    total = 0
    for num in args:
        total += num
    return total

print(sum_all_nums(1,2,3,4,5)) #how about 10 nums
print(sum_all_nums(1,2))
```

```
(1, 2, 3, 4, 5)
```

```
(1, 2)
```

```
15
```

```
3
```

**kwargs

- **kwargs คือ Parameter ที่ใช้แทน Parameter แบบมีชื่อ กี่ตัวก็ได้ มี Type เป็น Dictionary
- เราสามารถใช้ชื่อตัวแปรอื่นแทน kwargs ได้ เช่น **tmp, **vars แต่โดยทั่วไป programmer จะใช้ชื่อ **kwargs เป็นมาตรฐาน

**kwargs


```
def fav_colors(**kwargs):
    for person, color in kwargs.items():
        print(f'{person} -> {color}')
    print(kwargs)
```

```
dee -> pink
tip -> green
mock -> purple
pa -> yellow
```

```
{'dee': 'pink', 'tip': 'green', 'mock':
'purple', 'pa': 'yellow'}
```

```
fav_colors(dee = 'pink', tip='green', mock = 'purple', pa='yellow')
fav_colors('hi') #error
```

```
TypeError: fav_colors() takes 0 positional
arguments but 1 was given
```

**kwargs

คุณสามารถใช้ตัวดำเนินการ in เพื่อตรวจสอบว่ามี key อยู่ในพจนานุกรมหรือไม่

```
def special_greeting(**kwargs):
    if 'Dee' in kwargs and kwargs['Dee'] == 'special':
        return 'you get a special greeting'
    elif 'Dee' in kwargs:
        return 'Hello'
    return 'Not sure who this is...'

print(special_greeting(Dee='Hi'))
print(special_greeting(Dee='special'))
print(special_greeting(John='Yo'))
```

```
Hello
you get a special greeting
Not sure who this is...
```

Wrap-up all together

ถ้า function มีการรับ parameter ทั้ง 4 แบบ ลำดับของ parameters ควรมีการจัดวางดังนี้

1) parameters 2) *args 3) default parameters 4) **kwargs

Wrap-up all together

1) parameters 2) *args 3) default parameters 4) **kwargs

```
def display_info(a,b,*args, instructor='Mock', **kwargs):
    print(a)
    print(b)
    print(args)
    print(instructor)
    print(kwargs)
    return [a,b,args, instructor, kwargs]    #return a list


print(display_info(1,2,3,4,last_name='Yo', job='instructor'))
```

```
1
2
(3, 4)
Mock
{'last_name': 'Yo', 'job': 'instructor'}
```

```
[1, 2, (3, 4), 'Mock', {'last_name': 'Yo', 'job': 'instructor'}]
```

Contact: [jidapa.kra, akara.sup, suradej.int]@mahidol.edu

Wrap-up all together

~~1) parameters 2) *args 3) default parameters 4) **kwargs~~

```
def display_info(a,b,*args, instructor='Mock', **kwargs):
    print(a)
    print(b)
    print(args)
    print(instructor)
    print(kwargs)
    return [a,b,args, instructor, kwargs]   #return a list
```

What is the output?

```
print(display_info(1,2,3,4,instructor='Dee',last_name='Yo', job='instructor'))
print(display_info(1,2,3,4,'Dee',last_name='Yo', job='instructor'))
```

Lambda

Syntax `lambda parameters: expression`

- Lambda เป็น function อีกรูปแบบหนึ่งที่ไม่ต้องตั้งชื่อ (anonymous function)
- Lambda function จะ return ผลลัพธ์โดยอัตโนมัติ
- Lambda function มักถูกใช้ร่วมกับ built-in functions

```
def square(num):
    return num*num
print(square(9)) #81
```

```
square2= lambda num: num * num
print(square2(7)) #49
```

```
add = lambda a,b: a+b
print(add(3,10)) #13
```

```
cube = lambda num: num ** 3
print(cube(2))  #8
print(cube(3))  #27
```

Map

Syntax `map(function, iterable)`

- map เป็น built-in function ที่รับ 2 arguments คือ 1) a function และ 2) an iterable เช่น lists, string, dictionaries, sets, tuples
- map ทำงานโดยการเรียก function กับ element ทุกตัวใน an iterable และ return data structure ใหม่

```
nums = [2,4,6,8,10]
def double(x):
    return x*2
doubles = map(double, nums)
print(doubles)
print(list(doubles))
```

```
<map object at 0x118013a58>
```

```
[4, 8, 12, 16, 20]
```

Map

```
nums = [2,4,6,8,10]
doubles = map(lambda x: x*2,nums)
print(list(doubles))
```

```
[4, 8, 12, 16, 20]
```

```
people = ["Darcy", "Kitty", "Mark"]
peeps = map(lambda name: name.upper(), people)
print(list(peeps))
```

```
['DARCY', 'KITTY', 'MARK']
```

Map

```
names = [
    {"first":"Darcy", "last":"Rap"},
    {"first":"Kitty", "last":"Wang"},
    {"first":"Mark", "last":"Ton"}
    ]
first_name = list(map(lambda x: x['first'], names))
print(first_name)
```

```
['Darcy', 'Kitty', 'Mark']
```

Filter

Syntax `filter(function, iterable)`

- filter เป็น built-in function ที่รับ 2 arguments คือ 1) a function และ 2) an iterable เช่น lists, string, dictionaries, sets, tuples
- Map ทำงานโดยการเรียก function กับ element ทุกตัวใน an iterable และ return data ที่ให้ค่า True

```
list1 = [1,2,3,4]
evens =  list(filter(lambda x: x %2== 0, list1))
print(evens)
```

```
[2, 4]
```

```
names = ['austin', 'penny', 'anthony', 'angel', 'billy']
a_names = list(filter(lambda n: n[0]=='a', names))
print(a_names)
```

```
['austin', 'anthony', 'angel']
```

Filter/Map


```
users = [
            {"username": "samuel", "tweets": ["I love cake", "I love pie", "hello world!"]},
            {"username": "katie", "tweets": ["I love my cat"]},
            {"username": "jeff", "tweets": []},
            {"username": "bob123", "tweets": []},
            {"username": "doggo_luvr", "tweets": ["dogs are the best", "I'm hungry"]},
            {"username": "guitar_gal", "tweets": []}
]
#extract inactive users using filter:
inactive_users = list(filter(lambda u: len(u['tweets'])==0, users))
print(inactive_users)
# extract usernames of inactive users with map and filter:
usernames = list(map(lambda user: user["username"].upper(),
                 filter(lambda u: len(u['tweets'])==0, users)))
print(usernames)
```

```
[{'username': 'jeff', 'tweets': []},
{'username': 'bob123', 'tweets': []},
{'username': 'guitar_gal', 'tweets': []}]
```

```
['JEFF', 'BOB123', 'GUITAR_GAL']
```

Sorted

Syntax

```
sorted(iterable)
sorted(iterable, reverse=True)
```

- sorted เป็น built-in function ที่รับ an iterable เช่น lists, string, dictionaries, sets, tuples และ return data ที่เรียงจากน้อยไปมาก
- ถ้าต้องการเรียงลำดับข้อมูลจากมากไปน้อย ให้ระบุว่า reverse=True

```
nums = [4,6,1,30,55,23]
new_list = sorted(nums)
print(new_list)
```

```
[1, 4, 6, 23, 30, 55]
```

```
new_list = sorted(nums, reverse=True)
print(new_list)
```

```
[55, 30, 23, 6, 4, 1]
```

Max and min

Syntax `max(iterable)`

Syntax `min(iterable)`

- Max, min เป็น built-in function ที่รับ an iterable เช่น lists, string, dictionaries, sets, tuples หรือตัวเลขหลายตัว หรือตัวอักษรหลายตัว และ return data ที่มีค่ามากที่สุดหรือน้อยที่สุด

```
nums = [40,32,6,5,10]
print(max(3,67,99))                    #99
print(max('c','d','a'))                #d
print(max(nums))                       #40
print(min(3,67,99))                    #3
print(min('c','d','a'))                #a
print(min(nums))                       #5
```

reversed, abs, sum, round

Syntax `reversed(iterable)`

Syntax `abs(number)`

Syntax `sum(iterable)`

Syntax `round(number)`

```
nums = [40,32,6,5,10]
print(list(reversed(nums)))
print(list(reversed('hello')))
```

```
[10, 5, 6, 32, 40]
```

```
['o', 'l', 'l', 'e', 'h']
```

```
print(abs(-5))
print(abs(-3.444))
```

```
5
3.444
```

reversed, abs, sum, round


```
print(sum([1,2,3]))
print(sum([1,2,3], 10))
print(sum([1,2,3], -6))
print(sum((1.5,1.5,3.0)))
```

```
6
16
0
6.0
```

```
print(round(10.2))
print(round(3.51234, 3))
```

```
10
3.512
```

reversed, abs, sum, round

Methods	Description
`reversed(item)`	เรียงข้อมูลโดยเอาตัวท้ายขึ้นก่อน
`abs(item)`	คืนค่าจำนวนเต็มบวกของข้อมูลนั้น
`sum(item)`	คืนค่าผลรวมของข้อมูลนั้น
`round(item, decimal)`	ใช้เพื่อระบุจำนวนจุดทศนิยมที่ต้องการ ถ้าจำนวนจุดทศนิยมไม่ได้ถูกระบุ ตัวเลขจะถูกปัดให้เป็นจำนวนเต็มที่ใกล้เคียงที่สุด